

中国海洋大学计算机科学与技术学院

实验一 DataLab 实验报告

姓名：吴虹霖

年级：2024 级

专业：计算机科学与技术

科目：计算机系统基础

题目：实验 1 Data Lab

实验时间:2025.10.09

实验教师:蒋永国

一、实验目的：

1. 熟悉和掌握计算机中整数和浮点数的二进制编码表示
2. 实现简单的逻辑、二进制补码和浮点函数，但使用高度受限的 C 子集
3. 了解 C 数据类型的位级表示以及数据操作的位级行为

二、实验平台：

1. Educoder 在线实践教学平台

三、实验任务及要求：

实验要求

- 整数
 - 只能使用 0-255 的整型常数
 - 只能使用函数参数与函数内声明的局部变量
 - 只能使用如下单目操作符：! ~
 - 只能使用如下双目操作符：& ^ | + << >>
 - 最多只能使用有限个运算符（等于号、括号不计），可以认为使用的运算符个数越少得分越高
 - 一些函数可能对操作符有更多的限制（在题目前以操作符限制给出）
 - 禁止使用任何控制结构如 if do while for switch 等
 - 禁止定义或使用任何宏
 - 禁止定义任何函数
 - 禁止调用任何函数（除了可以使用 printf 输出中间变量，但提交时必须去掉）
 - 禁止使用任何形式的类型转换
 - 禁止使用 int 以外的任何类型（包括结构体、数组、联合体）
- 浮点数
 - 只能使用函数参数与函数内声明的局部变量
 - 最多只能使用有限个运算符（等于号、括号不计），可以认为使用的运算符个数越少得分越高
 - 禁止定义或使用任何宏
 - 禁止定义任何函数
 - 禁止调用任何函数（除了可以使用 printf 输出中间变量，但提交时必须去掉）
 - 禁止使用任何形式的类型转换

1.bitAnd:

补充函数 bitAnd(), 只用 ~、| 实现 $x \& y$, 将结果 return 返回。

1) 操作符限制: ~、|

2) 操作符使用数量限制: 8

2.getBytes:

本关任务: 补充函数 getByte(), 取出 x 中的 n 号字节, 将结果 return 返回。

1) 编号从低位到高位从 0 开始

2) 操作符使用数量限制: 6

3.logicalShift:

本关任务: 补充函数 logicalShift(), 将 x 逻辑右移 n 位 ($0 \leq n \leq 31$), 将结果 return 返回。

操作符使用数量限制: 20

bitCount:

本关任务: 补充函数 bitCount(), 统计 x 的二进制表示中 1 的数量, 将结果 return 返回。

操作符使用数量限制: 40

bang:

本关任务: 补充函数 bang(), 不使用 ! 实现 ! 操作符, 将结果 return 返回。

1) 操作符限制: ~ & ^ | + < >

2) 操作符使用数量限制: 12

tmin:

本关任务: 补充函数 tmin(), 返回补码表示的整型的最小值, 将结果 return 返回。

操作符使用数量限制: 4

7.fitsBits

本关任务: 补充函数 fitsBits(), 如果 x 可以只用 n 位补码表示则返回 1, 否则返回 0 ($1 \leq n \leq 32$)。

操作符使用数量限制: 15

8.divpwr2

本关任务: 补充函数 divpwr2(), 计算 $\lfloor x / 2^n \rfloor$, ($0 \leq n \leq 30$) 结果向零取整。

1) 操作符使用数量限制: 15

9.negate

本关任务: 补充函数 negate(), 计算 -x。

1) 操作符使用数量限制: 5

10.isPositive

本关任务: 补充函数 isPositive(), 如果 x 大于 0 返回 1, 否则返回 0。

1) 操作符使用数量限制: 8

11.isLessOrEqual

本关任务：补充函数 isLessOrEqual()，如果 x 小于等于 y 则返回 1，否则返回 0。

1)操作符使用数量限制：24

12.ilog2

本关任务：补充函数 ilog2()，返回 x 求以 2 为底的对数的结果（向下取整）。

1)操作符使用数量限制：90

13.float_neg

本关任务：补充函数 float_neg()，返回 -uf 的位级表示。

1)操作符使用数量限制：10

注意：

本题及以下所有的题目都采用 unsigned int 来存放位级表示

所有的浮点类型都为 float

如果输入为 NaN，返回 NaN

14.float_i2f

本关任务：补充函数 float_i2f()，实现由 int 到 float 的类型转换。

1)操作符使用数量限制：30

15.float_twice

本关任务：补充函数 float_twice()，返回 2*f 的位级表示。

1)操作符使用数量限制：30

四、实验过程及结果：

1、bitAnd:

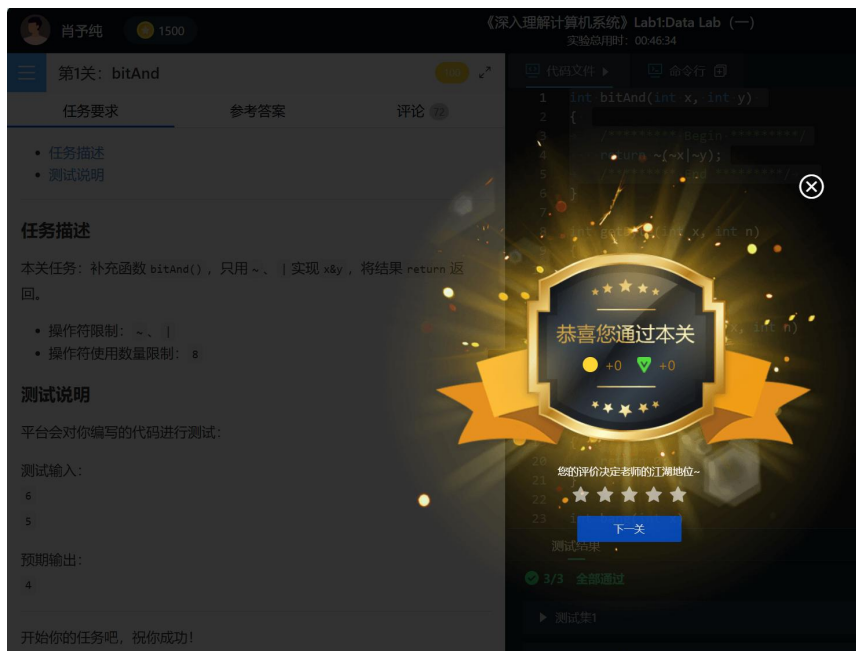
1) 实验代码：

```
int bitAnd(int x, int y)
{
    /****** Begin *****/
    return ~(~x|~y);
    /****** End *****/
}
```

1) 实验过程：

要实现 $x \& y$ ，相当于 $(x \text{ 的非}) + (y \text{ 的非})$ ，最后对此结果取反可得。

3) 实验结果：



2. getByte:

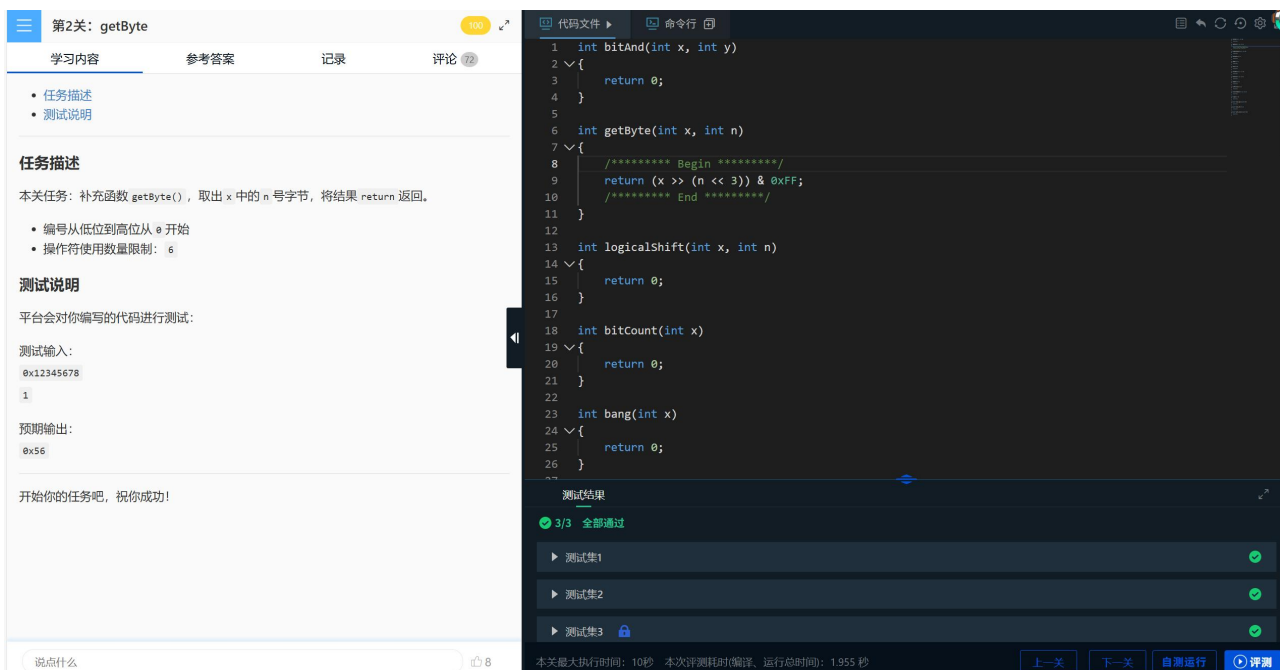
1) 实验代码:

```
1 int getByte(int x, int n)
2 {
3     /***** Begin *****/
4     return (x >> (n << 3)) & 0xFF;
5     /***** End *****/
6 }
```

2) 实验过程:

通过将 x 右移 $n \times 8$ 位使目标字节移至最低 8 位，再用 0xFF 掩码提取该字节值。

3) 实验结果:



3. logicalShift

1) 实验代码:

```
1 int logicalShift(int x, int n)
2 {
3     /***** Begin *****/
4     int mask = ~(((1 << 31) >> n) << 1);
5     return (x >> n) & mask;
6     /***** End *****/
7 }
```

实验过程：通过构造掩码清除算术右移带来的符号位扩展来实现逻辑右移功能。

实验结果：

The screenshot shows a coding platform interface for the 'logicalShift' task. The left sidebar contains the task description, which states the goal is to implement a logical right shift function. It also shows test input (0x87654321) and expected output (0x8765432). The main area displays the C code for the logicalShift function, which uses a mask to clear the sign bit extension. Below the code, the 'Test Results' section shows that all three test cases passed. The right sidebar shows the code for other functions: getByte, bitCount, bang, and tmin.

4. bitCount

1) 实验代码:

```
01 int bitCount(int x)
02 {
03     /***** Begin *****/
04     int mask1 = 0x55 | (0x55 << 8);
05     int mask2 = 0x33 | (0x33 << 8);
06     int mask3 = 0x0F | (0x0F << 8);
07     int mask4 = 0xFF | (0xFF << 16);
08     int mask5 = 0xFF | (0xFF << 8);
09
10     mask1 = mask1 | (mask1 << 16);
11     mask2 = mask2 | (mask2 << 16);
```

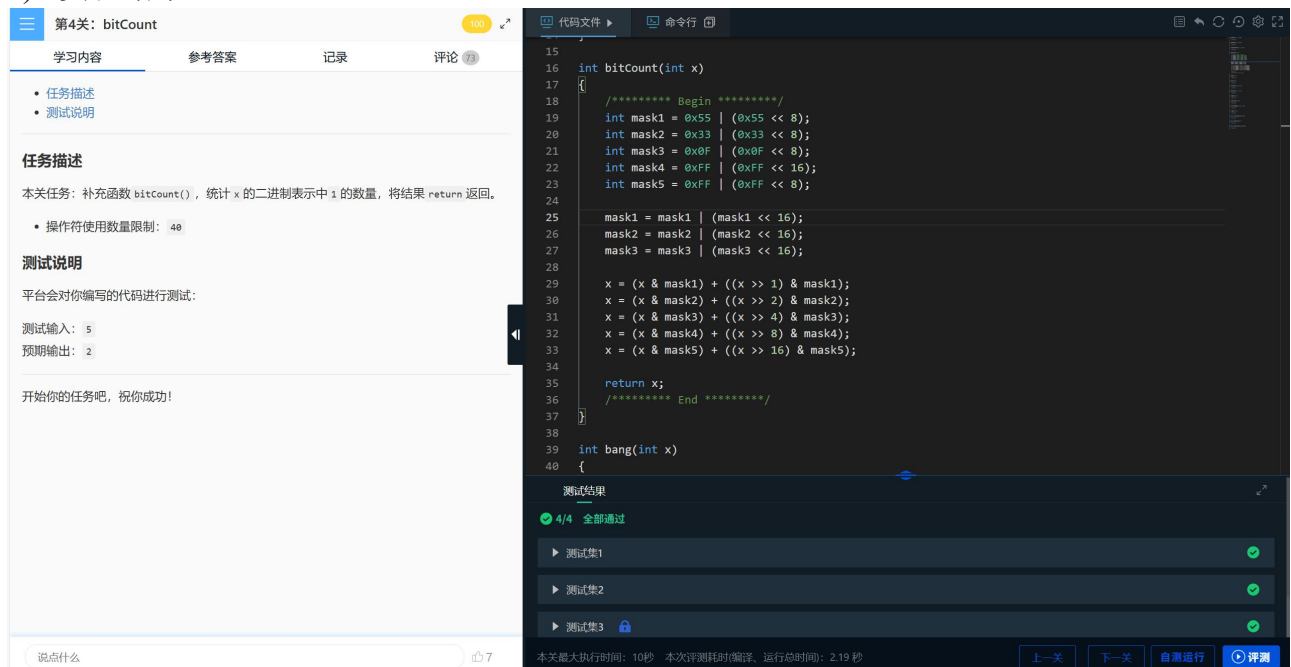
```

12     mask3 = mask3 | (mask3 << 16);
13
14     x = (x & mask1) + ((x >> 1) & mask1);
15     x = (x & mask2) + ((x >> 2) & mask2);
16     x = (x & mask3) + ((x >> 4) & mask3);
17     x = (x & mask4) + ((x >> 8) & mask4);
18     x = (x & mask5) + ((x >> 16) & mask5);
19
20     return x;
21     /***** End *****/
22 }

```

2) 实验过程：通过分治归并的位操作来统计二进制中 1 的个数，具体采用 5 次归并计算将相邻位的 1 数量相加，最终得到整个 32 位整数中 1 的总数。

3) 实验结果：



The screenshot displays a coding interface for a task titled '第4关: bitCount'. The left sidebar contains a '任务描述' (Task Description) section stating the goal is to complete the 'bitCount()' function to count the number of 1s in the binary representation of 'x'. Below this is a '测试说明' (Test Instructions) section with input/output examples and a success message. The main area shows the C code for 'bitCount' and 'bang' functions. The 'bitCount' function uses five masks and five shifts to count bits. The 'bang' function is partially visible. At the bottom, the '测试结果' (Test Results) section shows '4/4 全部通过' (All 4 tests passed) with individual test case status icons.

5. bang

1) 实验代码：

```

1 int bang(int x)
2 {
3     /***** Begin *****/
4     return ((x | (~x + 1)) >> 31) + 1;
5     /***** End *****/
6 }

```

2) 实验过程：利用 0 的补码特性来实现逻辑非功能：对于 0，其本身和补码的最高位都是 0，而对于非 0 数，其本身或补码的最高位必为 1，通过右移取符号位再加 1 来区分这两种情况。

3) 实验结果：

第5关: bang

100

学习内容

参考答案

记录

评论 73

任务描述

测试说明

任务描述

本关任务: 补充函数 `bang()`, 不使用 `!` 实现 `!` 操作符, 将结果 `return` 返回。

- 操作符限制: `~ & ^ | + << >>`
- 操作符使用数量限制: 12

测试说明

平台会对你编写的代码进行测试:

测试输入: 4

预期输出: 0

开始你的任务吧, 祝你成功!

说点什么

6

代码文件

命令行

```
8     return 0;
9 }
10
11 int logicalShift(int x, int n)
12 {
13     return 0;
14 }
15
16 int bitCount(int x)
17 {
18     return 0;
19 }
20
21 int bang(int x)
22 {
23     /****** Begin *****/
24     return ((x | (~x + 1)) >> 31) + 1;
25     /****** End *****/
26 }
27
28 int tmin(void)
29 {
30     return 0;
31 }
32
33 int fitsBits(int x, int n)
```

测试结果

3/3 全部通过

测试集1

测试集2

测试集3

本关最大执行时间: 10秒 本次评测耗时(编译、运行总时间): 2.176 秒

上一关 下一关 自动运行 评测

6. tmin

1) 实验代码:

```
1 int tmin(void)
2 {
3     /****** Begin *****/
4     return 1 << 31;
5     /****** End *****/
6 }
```

1) 实验过程: 将 1 左移 31 位来直接得到补码表示的最小整数值, 即最高位为 1、其余位为 0 的二进制数。

2) 实验结果:

第6关: tmin

100

学习内容

参考答案

记录

评论 5

任务描述

测试说明

任务描述

本关任务: 补充函数 `tmin()`, 返回补码表示的整型的最小值, 将结果 `return` 返回。

- 操作符使用数量限制: 4

测试说明

平台会对你编写的代码进行测试:

测试输入: 无

预期输出: 整型的最小值

开始你的任务吧, 祝你成功!

说点什么

5

代码文件

命令行

```
10
11 int logicalShift(int x, int n)
12 {
13     return 0;
14 }
15
16 int bitCount(int x)
17 {
18     return 0;
19 }
20
21 int bang(int x)
22 {
23     return 0;
24 }
25
26 int tmin(void)
27 {
28     /****** Begin *****/
29     return 1 << 31;
30     /****** End *****/
31 }
32
33 int fitsBits(int x, int n)
34 {
35     return 0;
36 }
```

测试结果

1/1 全部通过

测试集1

本关最大执行时间: 10秒 本次评测耗时(编译、运行总时间): 0.778 秒

上一关 下一关 自动运行 评测

7.fitsBits

1) 实验代码:

```
1 int fitsBits(int x, int n)
2 {
3     /***** Begin *****/
4     int shift = 32 + ~n + 1; // 32 - n
5     int shifted = (x << shift) >> shift;
6     return !(x ^ shifted);
7     /***** End *****/
8 }
```

1) 实验过程: 如果 x 可以用 n 位补码表示, 那么将 x 左移 $(32-n)$ 位再算术右移 $(32-n)$ 位后应该等于原值 x

2) 实验结果:

The screenshot displays a coding environment with two main panels. The left panel, titled '第7关: fitsBits', contains the task description and test results. The task description states: '补充函数 fitsBits(), 如果 x 可以用 n 位补码表示则返回 1, 否则返回 0 ($1 \leq n \leq 32$)。' The test results section shows '测试输入: 5, 4' and '预期输出: 1'. The right panel shows the code editor with the implementation of fitsBits and other functions: bang, tmin, divpwr2, and negate. The fitsBits function is implemented as follows:

```
21 int bang(int x)
22 {
23     return 0;
24 }
25
26 int tmin(void)
27 {
28     return 0;
29 }
30 int fitsBits(int x, int n)
31 {
32     /***** Begin *****/
33     int shift = 32 + ~n + 1; // 32 - n
34     int shifted = (x << shift) >> shift;
35     return !(x ^ shifted);
36     /***** End *****/
37 }
38
39 int divpwr2(int x, int n)
40 {
41     return 0;
42 }
43
44 int negate(int x)
45 {
46     return 0;
47 }
```

At the bottom, the '测试结果' (Test Results) section shows '4/4 全部通过' (All passed) for three test sets.

3)

8.divpwr2

1) 实验代码:

```
1 int divpwr2(int x, int n)
2 {
3     /***** Begin *****/
4     int sign = x >> 31;
5     return (x + (sign & ((1 << n) - 1))) >> n;
6     /***** End *****/
7 }
```

实验过程: 实现整数除以 2 的 n 次幂, 并向零取整。对于正数, 直接右移 n 位即可; 对于负数, 需要特殊处理以保证向零取整。

实验结果:

第8关: divpwr2

100

学习内容

参考答案

记录

评论 73

- 任务描述
- 测试说明

任务描述

本关任务：补充函数 `divpwr2()`，计算 $\lfloor \frac{x}{2^n} \rfloor$ ，($0 \leq n \leq 30$) 结果向零取整。

- 操作符使用数量限制：15

测试说明

平台会对你编写的代码进行测试：

测试输入：

```
20
2
```

预期输出：

```
5
```

开始你的任务吧，祝你成功！

代码文件

命令行

```

19 }
20
21 int bang(int x)
22 {
23     return 0;
24 }
25
26 int tmin(void)
27 {
28     return 0;
29 }
30
31 int fitsBits(int x, int n)
32 {
33     return 0;
34 }
35
36 int divpwr2(int x, int n)
37 {
38     /***** Begin *****/
39     int sign = x >> 31;
40     return (x + (sign & ((1 << n) - 1))) >> n;
41     /***** End *****/
42 }
43
44 int negate(int x)
45 {

```

测试结果

3/3 全部通过

测试集1

测试集2

测试集3

本文最大执行时间：10秒 本次评测耗时(编译、运行总时间)：2.087 秒

上一关

下一关

自测运行

评测

9. negate

1) 实验代码：

```

1 int negate(int x)
2 {
3     /***** Begin *****/
4     return ~x + 1;
5     /***** End *****/
6 }

```

实验过程：实现计算 $-x$ 的函数。在二进制补码表示中，一个数的负数可以通过取反加一得到。

实验结果：

第9关: negate

100

学习内容

参考答案

记录

评论 73

- 任务描述
- 测试说明

任务描述

本关任务：补充函数 `negate()`，计算 $-x$ 。

- 操作符使用数量限制：5

测试说明

平台会对你编写的代码进行测试：

测试输入：

```
5
```

预期输出：

```
-5
```

开始你的任务吧，祝你成功！

代码文件

命令行

测试结果

4/4 全部通过

测试集1

测试集2

测试集3

本文最大执行时间：10秒 本次评测耗时(编译、运行总时间)：2.807 秒

上一关

下一关

自测运行

评测

10.isPositive

1) 实验代码:

```
1 int isPositive(int x)
2 {
3     /***** Begin *****/
4     int sign = x >> 31;    /
5     return !sign & (!!x);
6     /***** End *****/
7 }
```

实验过程:

将 x 算术右移 31 位, 对于负数会得到全 1 (-1), 对于正数会得到全 0

对上面的结果取逻辑非, 对于负数得到 0, 对于正数得到 1

对 x 取两次逻辑非, 如果 x 为 0 则结果为 0, 如果 x 不为 0 则结果为 1

将两者进行与操作: 只有当 x 是正数 (符号位为 0 且不为 0) 时, 结果才为 1

实验结果:

第10关: isPositive

学习内容 参考答案 记录 评论 (7)

- 任务描述
- 测试说明

任务描述

本关任务: 补充函数 isPositive(), 如果 x 大于 0 返回 1, 否则返回 0。

- 操作符使用数量限制: 8

测试说明

平台会对你编写的代码进行测试:

测试输入: 4
预期输出: 1

开始你的任务吧, 祝你成功!

说点什么

5

```
35
36 int divpwr2(int x, int n)
37 {
38     return 0;
39 }
40
41 int negate(int x)
42 {
43     return 0;
44 }
45
46 int isPositive(int x)
47 {
48     /***** Begin *****/
49     int sign = x >> 31;    /
50     return !sign & (!!x);
51     /***** End *****/
52 }
53
54 int isLessOrEqual(int x, int y)
55 {
56     return 0;
57 }
58
59 int ilog2(int x)
60 {
61     return 0;
62 }
```

测试结果

4/4 全部通过

- 测试集1
- 测试集2
- 测试集3

本关最大执行时间: 10秒 本次评测耗时: 编译、运行总时间: 2.672 秒

上一关 下一关 自动运行 评测

11.isLessOrEqual

1) 实验代码:

```
1 int isLessOrEqual(int x, int y)
2 {
3     /***** Begin *****/
4
5     int sign_diff = ((x ^ y) >> 31) & 1; // 符号是否不同
6     return (sign_diff & (x >> 31)) | (!sign_diff & !((y + (~x
7 + 1)) >> 31));
8     /***** End *****/
9 }
```

2) 实验过程:

如果 x 和 y 符号不同, 且 x 为负数, 则返回 1

如果 x 和 y 符号相同, 且 $y-x$ 非负, 则返回 1

实验结果:

The screenshot shows a coding challenge interface for the problem 'isLessOrEqual'. On the left, the '任务描述' (Task Description) and '测试说明' (Test Description) are visible. The task is to implement the `isLessOrEqual` function. The test description states that the platform will test the code. The test input is 5 and 8, and the expected output is 1. On the right, the code editor shows the implementation of `isLessOrEqual` and other helper functions like `negate`, `isPositive`, and `ilog2`. The test results at the bottom show that all 4 test cases passed.

第11关: isLessOrEqual

学习内容 参考答案 记录 评论 (7)

任务描述

本关任务: 补充函数 `isLessOrEqual()`, 如果 x 小于等于 y 则返回 1, 否则返回 0。

操作符使用数量限制: 24

测试说明

平台会对你编写的代码进行测试:

测试输入:

5
8

预期输出:

1

开始你的任务吧, 祝你成功!

说点什么

代码文件 命令行

```
37 {  
38     return 0;  
39 }  
40  
41 int negate(int x)  
42 {  
43     return 0;  
44 }  
45  
46 int isPositive(int x)  
47 {  
48     return 0;  
49 }  
50  
51 int isLessOrEqual(int x, int y)  
52 {  
53     /***** Begin *****/  
54  
55     int sign_diff = ((x ^ y) >> 31) & 1; // 符号是否不同  
56     return (sign_diff & (x >> 31)) | (!sign_diff & !((y + (~x + 1)) >> 31));  
57     /***** End *****/  
58 }  
59  
60 int ilog2(int x)  
61 {  
62     return 0;  
63 }
```

测试结果

4/4 全部通过

测试集1 测试集2 测试集3

本关最大执行时间: 10秒 本次评测耗时(编译、运行总时间): 2.942 秒

上一关 下一关 自动运行 评测

12.ilog2

1) 实验代码:

```
01 int ilog2(int x)  
02 {  
03     /***** Begin *****/  
04     // 使用二分查找法找到最高位 1 的位置  
05     int shift, result = 0;  
06  
07     // 检查高 16 位是否有 1  
08     shift = (!!(x >> 16)) << 4;  
09     result |= shift;  
10     x >>= shift;  
11  
12     // 检查接下来的 8 位  
13     shift = (!!(x >> 8)) << 3;  
14     result |= shift;  
15     x >>= shift;  
16  
17     // 检查接下来的 4 位  
18     shift = (!!(x >> 4)) << 2;  
19     result |= shift;  
20     x >>= shift;
```

```

21
22 // 检查接下来的 2 位
23 shift = (!(x >> 2)) << 1;
24 result |= shift;
25 x >>= shift;
26
27 // 检查最后 1 位
28 shift = (!(x >> 1));
29 result |= shift;
30
31 return result;
32 /***** End *****/
33 }

```

实验过程：使用二分法查找最高有效位的位置

实验结果：

The screenshot displays a coding platform interface for a task titled "第12关: ilog2". The left sidebar contains the task description and test instructions. The main area shows the C code implementation of the `ilog2` function, which uses a binary search method to find the position of the highest set bit. The right sidebar shows the test results, indicating that all 4/4 test cases passed.

任务描述
 本关任务：补充函数 `ilog2()`，返回 `x` 求以 2 为底的对数的结果（向下取整）。

测试说明
 平台会对你编写的代码进行测试：
 测试输入：64
 预期输出：6
 开始你的任务吧，祝你成功！

```

55
56 int ilog2(int x)
57 {
58     /***** Begin *****/
59     // 使用二分查找法找到最高位1的位置
60     int shift, result = 0;
61
62     // 检查高16位是否有1
63     shift = (!(x >> 16)) << 4;
64     result |= shift;
65     x >>= shift;
66
67     // 检查接下来的8位
68     shift = (!(x >> 8)) << 3;
69     result |= shift;
70     x >>= shift;
71
72     // 检查接下来的4位
73     shift = (!(x >> 4)) << 2;
74     result |= shift;
75     x >>= shift;
76
77     // 检查接下来的2位
78     shift = (!(x >> 2)) << 1;
79     result |= shift;
80     x >>= shift;

```

测试结果
 4/4 全部通过
 测试集1 通过
 测试集2 通过
 测试集3 通过
 本文最大执行时间：10秒 本次评测耗时(编译、运行总时间)：2.862 秒
 上一关 自测运行 评测

13.float_neg

1) 实验代码：

```

1 unsigned float_neg(unsigned uf)
2 {
3     /***** Begin *****/
4     if (((uf & 0x7F800000) == 0x7F800000) && (uf &
5     0x007FFFFF)) {
6         return uf;
7     }
8     return uf ^ 0x80000000;
9     /***** End *****/

```

```
}  
}
```

2) 实验过程：检查指数位是否全为 1 (0x7F800000 掩码) 且尾数位非 0。如果是 NaN，返回原值。否则翻转最高位 (符号位)

3) 实验结果：

第1关: float_neg

学习内容 参考答案 记录 评论

- 任务描述
- 测试说明

任务描述

本关任务：补充函数 float_neg(), 返回 -uf 的位级表示。

- 操作符使用数量限制：10

注意：

- 本题及以下所有的题目都采用 unsigned int 来存放位级表示
- 所有的浮点类型都为 float
- 如果输入为 NaN，返回 NaN

测试说明

平台会对你编写的代码进行测试：

测试输入：-111
预期输出：0xffffffff91

测试输入：-12345
预期输出：0xffffcfc7

开始你的任务吧，祝你成功!

说点什么 10

代码文件

```
52 {  
53     return 0;  
54 }  
55  
56 int ilog2(int x)  
57 {  
58     return 0;  
59 }  
60  
61 unsigned float_neg(unsigned uf)  
62 {  
63     /***** Begin *****/  
64     if (((uf & 0x7F800000) == 0x7F800000) && (uf & 0x007FFFFF)) {  
65         return uf;  
66     }  
67     return uf ^ 0x80000000;  
68     /***** End *****/  
69 }  
70  
71 unsigned float_izf(int x)  
72 {  
73     return 0;  
74 }  
75  
76 unsigned float_twice(unsigned uf)  
77 {  
78     return 0;  
79 }
```

测试结果

5/5 全部通过

- 测试集1
- 测试集2
- 测试集3

本文最大执行时间：10秒 本次评测耗时(编译、运行总时间)：2.846 秒

下一关 自动运行 评测

14.float_izf

1) 实验代码：

```
01 unsigned float_izf(int x)  
02 {  
03     /***** Begin *****/  
04     if (x == 0) return 0;  
05     unsigned sign = 0, absx, frac, round_mask, round_part;  
06     int shift = 0, exp;  
07  
08     if (x < 0) {  
09         sign = 1 << 31;  
10         absx = -x;  
11     } else {  
12         absx = x;  
13     }  
14  
15     // 找到最高位 1 的位置  
16     unsigned tmp = absx;  
17     while (tmp >> 1) {  
18         tmp >>= 1;  
19         shift++;  
20     }  
21 }
```

```

22     exp = shift + 127; // 指数部分
23     if (shift <= 23) {
24         frac = (absx << (23 - shift)) & 0x7FFFFFFF;
25     } else {
26         int extra = shift - 23;
27         unsigned mask = (1 << extra) - 1;
28         unsigned remainder = absx & mask;
29         frac = (absx >> extra) & 0x7FFFFFFF;
30         unsigned half = 1 << (extra - 1);
31         // 四舍五入到偶数
32         if (remainder > half || (remainder == half && (frac & 1))) {
33             frac++;
34             if (frac >> 23) {
35                 frac &= 0x7FFFFFFF;
36                 exp++;
37             }
38         }
39     }
40
41     return sign | (exp << 23) | frac;
42     /***** End *****/
43 }

```

实验过程：将整数分解为符号、指数和尾数三部分，通过位操作和舍入处理组合成 IEEE 754 浮点数格式。

实验结果：

The screenshot displays a C programming exercise titled '第2关: float_i2f'. The task description on the left states: '补充函数 float_i2f(), 实现由 int 到 float 的类型转换。' (Complete the function float_i2f(), implementing the type conversion from int to float). It also mentions a constraint: '操作符使用数量限制: 30' (Operator usage limit: 30). The test cases show an input of 3510593 and an expected output of 0x4a564504. The code editor on the right shows the implementation of the float_i2f function, which uses bit manipulation to convert an integer to a float. The test results at the bottom show that all three test cases passed, with a total score of 3/3.

15.float_twice

1) 实验代码:

```
01 unsigned float_i2f(int x)
```

```

02  {
03      /***** Begin *****/
04      if (x == 0) return 0;
05      unsigned sign = 0, absx, frac, round_mask, round_part;
06      int shift = 0, exp;
07
08      if (x < 0) {
09          sign = 1 << 31;
10          absx = -x;
11      } else {
12          absx = x;
13      }
14
15      // 找到最高位 1 的位置
16      unsigned tmp = absx;
17      while (tmp >> 1) {
18          tmp >>= 1;
19          shift++;
20      }
21
22      exp = shift + 127; // 指数部分
23      if (shift <= 23) {
24          frac = (absx << (23 - shift)) & 0x7FFFFFFF;
25      } else {
26          int extra = shift - 23;
27          unsigned mask = (1 << extra) - 1;
28          unsigned remainder = absx & mask;
29          frac = (absx >> extra) & 0x7FFFFFFF;
30          unsigned half = 1 << (extra - 1);
31          // 四舍五入到偶数
32          if (remainder > half || (remainder == half && (frac & 1))) {
33              frac++;
34              if (frac >> 23) {
35                  frac &= 0x7FFFFFFF;
36                  exp++;
37              }
38          }
39      }
40
41      return sign | (exp << 23) | frac;
42      /***** End *****/
43  }

```

实验过程：通过分别处理非规格化数（尾数左移）和规格化数（指数加 1）来实现浮点数乘以 2 的功能。

实验结果：

The screenshot shows a web-based coding challenge interface. On the left, the task description for '第3关: float_twice' is displayed, including a task description, test instructions, and test input/output. The main area shows the C code for the function, which manipulates the IEEE 754 floating-point format by shifting the exponent and fraction. The right sidebar shows the test results, indicating that all three test cases passed.

任务描述

本关任务：补充函数 `float_twice()`，返回 2^f 的位级表示。

• 操作符使用数量限制：30

测试说明

平台会对你编写的代码进行测试：

测试输入：111
预期输出：0x40000000

开始你的任务吧，祝你成功！

```
77 unsigned frac = uf & 0x007FFFFF;
78
79 // 如果是 NaN 或无穷大，直接返回原值
80 if (exp == 0x7F800000) {
81     return uf;
82 }
83
84
85 if (exp == 0) {
86     frac <<= 1;
87
88     if (frac & 0x00800000) {
89         exp = 0x00800000;
90         frac &= 0x007FFFFF;
91     }
92     return sign | exp | frac;
93 }
94
95 exp += 0x00800000;
96
97 if (exp == 0x7F800000) {
98     frac = 0; // 无穷大的尾数为0
99 }
100
101 return sign | exp | frac;
102
```

测试结果

3/3 全部通过

- 测试集1
- 测试集2
- 测试集3

本关最大执行时间：10秒 本次评测耗时(编译、运行总时间)：1.928 秒

五、实验总结

本次实验挑战我们使用高度受限的 C 语言子集（仅包含位运算和直线代码）来实现基础函数。在完成诸如计算绝对值、判断奇偶等任务的过程中，我们被迫跳出常规编程思维，直接与数据的二进制位“对话”。起初，这种抽象的操作让人感到困惑，但通过反复调试和观察，我们清晰地看到了整数补码、浮点数格式在内存中的真实样貌。这次实践让我深刻体会到数据在计算机中的本质表示方式，以及运算背后的位级行为，将课本上抽象的理论知识转化为了牢固的、直观的理解，收获极大。